ISM6562/4545: Big Data for AI & Analytics
Final Project: SupplyChain Manufacturing

The Leftside Undergrads

Aisha Nomani
An Chu
Aurelia Vo
Sai Han Shan Pha
Mohammed Aljumai

Due Date: May 6, 2026
Professor: Dr. Timothy Smith

Executive Summary

This project focuses on designing a big data pipeline for PrecisionParts Manufacturing to improve how the company uses its data for decision-making. Currently, the company is facing multiple challenges, including increasing defect rates, inefficient inventory management, supplier performance issues, and a lack of visibility across production, quality, and equipment data. Because these data sources are not connected, it is difficult for managers to identify problems early or understand the root causes of issues.

To address these problems, our team developed a scalable data pipeline that integrates data from production lines, quality inspections, inventory systems, supplier records, and equipment sensors. The system is built using a data lake architecture with three main zones: landing, curated, and analytics. The landing zone stores raw data in its original format, ensuring data integrity. The curated zone cleans, standardizes, and transforms the data into Parquet format for better performance.

The analytics zone contains final aggregated datasets that support reporting and business insights. Apache Spark is used as the main processing engine for batch transformations, while Kafka and Spark Structured Streaming are used for near real-time processing of equipment sensor data. Apache Airflow is used to automate and orchestrate the pipeline, ensuring reliability and proper scheduling.

Through this system, the company is now able to connect previously separate datasets and generate meaningful insights. For example, the pipeline identifies which factories, production lines, and shifts have higher defect rates and shows that night shifts tend to have more defects. It also evaluates supplier performance by linking supplier data with production outcomes, helping the company identify high-risk suppliers.

In addition, the system detects inventory risks by identifying products that are below reorder levels and may lead to stockouts. The near real-time streaming component allows the company to monitor machine conditions and detect abnormal temperature or vibration patterns, supporting predictive maintenance and reducing downtime.

Overall, this project provides PrecisionParts with a unified and scalable data platform that improves visibility across operations. It helps the company reduce defects, improve supplier decisions, optimize inventory, and prevent equipment failures. By using data more effectively, PrecisionParts can reduce costs, increase efficiency, and make faster, more informed business decisions.

In the future, this pipeline can be expanded to support more advanced analytics and machine learning models, such as defect prediction, demand forecasting, and automated decision systems. As the company grows, the scalable design of the pipeline allows it to handle larger data volumes without major changes. This makes the system not only useful for current operations but also valuable for long-term strategic planning and continuous improvement.

Architecture Design

Our PrecisionParts Manufacturing project is built as a structured big data pipeline that transforms raw manufacturing data into valuable insights for PrecisionParts. The goal of this architecture isn't just to simply move data through different stages, but rather to directly address the company's main challenges, which include but are not limited to rising defect rates, inventory inefficiencies, and limited visibility across production, supplier, and equipment data. The design focuses on creating a system where these different data sources can be connected and analyzed to help make more informed decisions.

The pipeline uses an HDFS-style data lake architecture divided into three zones: landing, curated, and analytics. The landing zone stores raw datasets exactly as they are received, such as compressed CSV and JSON files. This preserves the original data and allows it to be reprocessed if necessary. The curated zone is where the data is cleaned, standardized, and converted into Parquet format to improve processing efficiency. The analytics zone contains final aggregated datasets that support reporting and help answer key business questions such as defect trends, supplier performance, and inventory levels.

The data flows through these zones in a structured way. Raw data is first stored in the landing zone, then processed in Spark where it is cleaned and validated. After that, multiple datasets are joined together to create a more complete view of operations. For example, production data is linked with quality inspection data to understand defect causes, and inventory data is connected with supplier performance data to identify supply chain inefficiencies. These transformations allow the pipeline to support important business questions, including defect prediction, supplier quality analysis, and inventory optimization. The processed data is stored in the curated zone, and final aggregated results are written to the analytics zone.

The technology choices are based on scalability and performance. HDFS is used for storage because it supports distributed data, while Apache Spark is used for processing due to its ability to efficiently handle large-scale transformations and joins. Parquet is used in the curated and analytics zones because its column-based format improves performance by reducing the amount of data that needs to be read. The raw data remains in its original format in the landing zone to preserve data integrity.

The architecture also includes a streaming component for real-time processing. Equipment sensor data is continuously generated by machines, making it suitable for streaming. Using Kafka and Spark Structured Streaming, this data can be processed in real-time to detect unusual patterns such as abnormal temperature or vibration levels. This supports predictive maintenance and helps reduce unplanned downtime. Airflow is used as the orchestration tool to manage scheduling, ensure tasks run in the correct order, and improve the reliability of the pipeline.

All in all, the architecture integrates previously disconnected data sources into a single system that supports both batch analytics and real-time monitoring. By separating raw, processed, and analytical data, the pipeline remains organized, scalable, and easier to maintain. This structure allows PrecisionParts to identify root causes of defects, improve supplier evaluation, and make more informed inventory decisions as data continues to grow.

Implementation

The implementation of the PrecisionParts Manufacturing big data pipeline focuses on building a scalable and fault-tolerant architecture capable of supporting both historical analytics and real-time monitoring. The system integrates Apache Spark for batch processing, Apache Kafka and Spark Structured Streaming for real-time data ingestion and processing, and Apache Airflow for workflow orchestration. Throughout the implementation, key technical decisions were made to balance scalability, performance, and reliability under increasing data volume and operational complexity.

A central architectural decision in this project was the adoption of a Lambda Architecture, which separates batch processing and streaming processing into two complementary layers. The batch layer is responsible for computationally intensive tasks such as cross-domain joins, defect rate aggregation, supplier analysis, and inventory risk computation. These operations require full dataset access and are best suited for distributed batch processing using Spark. In contrast, the streaming layer is designed for low-latency processing of machine sensor data to enable real-time anomaly detection. This hybrid design was chosen over a pure streaming (Kappa) architecture because the dataset required complex joins across multiple enterprise systems, which are inefficient to maintain in a continuous streaming-only environment. At the same time, real-time monitoring of equipment conditions required immediate processing, making the dual-layer approach essential.

Another important design decision was the selection of Parquet as the primary storage format for both the curated and analytics zones. Parquet was chosen due to its columnar structure, which significantly improves query performance by allowing Spark to read only required columns instead of scanning entire datasets. This reduces both I/O cost and processing time, especially for large-scale analytical queries. While alternative formats such as Avro were evaluated for their schema evolution capabilities, the datasets in this project had stable schemas, making Parquet the more efficient choice for analytical workloads. HDFS was used as the underlying storage system, structured into a multi-zone data lake architecture consisting of landing, curated, and analytics layers. The landing zone stores raw data in its original format, ensuring traceability and reproducibility. The curated zone contains cleaned and standardized datasets, while the analytics zone stores final aggregated outputs used for reporting and decision-making.

The batch processing pipeline was implemented using Apache Spark and is responsible for transforming raw data into structured analytical datasets. The pipeline begins with a comprehensive data cleaning phase applied consistently across all datasets. This includes removing duplicate records, handling missing values in key fields, and explicitly casting data types for numerical and timestamp columns. In addition, sensor data undergoes strict validation to remove physically impossible values, such as negative vibration or unrealistic temperature readings. Supplier performance metrics are also normalized to ensure consistency across different formats. These preprocessing steps are essential for maintaining data integrity before performing any analytical operations.

Once the data is cleaned, cross-domain integrations are performed to enable enterprise-wide analytics. The production dataset is joined with quality inspection data to calculate defect rates across factories, production lines, and shifts. This integration enables the identification of operational patterns that contribute to manufacturing defects. In addition, production data is joined with supplier performance

data to evaluate the relationship between supplier quality and downstream production outcomes. This allows the system to assess whether supplier-reported metrics align with actual production defects. Equipment sensor data is aggregated at the factory and daily level and then joined with production data to analyze the impact of machine conditions on production efficiency and quality outcomes. These joins are computationally expensive but essential for enabling root-cause analysis across previously siloed datasets.

The streaming component of the system is implemented using Apache Kafka and Spark Structured Streaming. Kafka serves as the ingestion layer for real-time machine sensor data, which is published to a single topic named machine-telemetry. Each message in this topic represents a structured event containing machine identifiers, timestamps, and sensor readings such as temperature, vibration, power consumption, and pressure. A key design decision was to use machine_id as the partition key in Kafka to ensure that all events from a single machine are processed in order. This is critical for maintaining time-series consistency during downstream processing.

Spark Structured Streaming consumes data from Kafka and processes it using sliding time windows. The system aggregates sensor data into one-minute windows with updates every thirty seconds, allowing near real-time monitoring of machine behavior. A watermarking strategy is implemented to handle late-arriving data while preventing unbounded state growth in memory. Instead of using static thresholds for anomaly detection, the system computes machine-specific baselines based on historical behavior. This approach significantly improves accuracy by accounting for differences in operating conditions across machines. Alerts are generated when sensor values exceed dynamically computed thresholds, categorized into WARNING and CRITICAL levels. These alerts are written to Parquet files and also displayed in real-time for monitoring purposes.

Workflow orchestration is handled using Apache Airflow, which manages both batch and streaming-related monitoring processes. The batch pipeline is structured as a sequence of dependent tasks, beginning with data availability validation in HDFS. This step ensures that all required datasets exist before processing begins. The next stage executes Spark jobs for data cleaning and transformation, followed by analytical processing that generates aggregated outputs. A final reporting step produces summary outputs for dashboard consumption. The entire pipeline is configured with retry mechanisms to handle transient failures, and all tasks are designed to be idempotent by using overwrite mode in Spark writes. This ensures that re-running the pipeline does not produce duplicate or inconsistent results.

In addition to the batch pipeline, a separate Airflow DAG is implemented to monitor the health of the streaming system. This monitoring pipeline runs at regular intervals and checks whether new streaming outputs are being generated in the expected output directory. If no new data is detected within a defined time threshold, the system triggers an alert, indicating potential failures in the streaming pipeline. This separation of concerns ensures that streaming health monitoring does not interfere with data processing workloads.

A critical component of the implementation is the data quality strategy, which is applied at multiple stages of the pipeline. At the file level, Airflow verifies that all required input files exist and are non-empty before execution begins. At the record level, validation rules ensure that cleaned datasets maintain expected row counts within an acceptable tolerance range, typically within five percent of raw data volumes. Additional checks are applied to filter invalid or corrupted records, particularly in sensor data where physical constraints must be enforced. At the referential level, cross-dataset validation ensures

that key identifiers such as batch_id exist across related datasets. Any records that fail these checks are not discarded but instead moved to a quarantine area for further inspection, ensuring full traceability and preventing silent data loss.

Several implementation challenges were encountered during development. One major challenge was the integration of multiple heterogeneous datasets originating from different operational systems, which had inconsistent formats and schemas. This was resolved by enforcing a standardized schema during the Spark cleaning phase. Another challenge was performance degradation caused by large cross-domain joins, which was addressed through partitioning strategies and the use of Parquet to reduce I/O overhead. In the streaming layer, managing state size for continuous sensor ingestion was a challenge, which was solved using watermarking to limit memory growth. Finally, ensuring pipeline reliability in a local environment required careful design of retry mechanisms and idempotent processing to prevent failures from corrupting outputs.

Overall, the implementation successfully integrates batch processing, real-time streaming, and orchestration into a unified big data system. The architecture is designed to be modular, scalable, and fault-tolerant, enabling it to handle both historical analysis and real-time operational monitoring. It also provides a clear path for future migration to cloud platforms such as AWS or Databricks with minimal code changes, reinforcing its scalability and production-readiness.

Business Insights

The PrecisionParts manufacturing analytics pipeline produced several concrete, data-driven findings across defect quality, supplier performance, inventory management, and equipment health. These insights directly address the operational challenges the company identified at the start of the project.

Defect Analysis and Root Cause Identification

The batch transformation pipeline computed defect rates across all eight factories, broken down by production line, shift, and operator. This cross-domain analysis was not possible before the project because quality inspection data and production line data had never been joined together. By linking these datasets on batch ID and product ID, the pipeline revealed which specific combinations of factory, shift, and production line were responsible for the highest defect concentrations.

The analysis confirmed what the VP of Quality had long suspected: defect rates are not uniformly distributed. Certain shifts and production lines consistently showed elevated defect rates compared to company-wide averages. Night shifts in particular showed higher defect rates across multiple factories, which suggests that staffing, fatigue, or reduced supervision may be contributing factors worth investigating. The pipeline quantified these differences for the first time, giving leadership a data-backed basis for targeted interventions such as additional training or adjusted staffing patterns rather than broad company-wide responses.

The defect code analysis also revealed which defect types occurred most frequently and which carried the highest severity. Critical severity defects, which trigger $2,500 containment charges when they reach customer assembly lines, were concentrated in a small number of defect categories. This means that addressing just a handful of root causes has the potential to eliminate a large share of the company's quality costs.

Supplier Quality and Delivery Performance

The supplier scorecard aggregation connected upstream supplier quality scores to downstream production defect rates for the first time. The pipeline joined supplier performance records with production line data on supplier ID, enabling a direct comparison between a supplier's reported quality score and the actual defect rate observed on production lines using that supplier's materials.

The results showed that supplier quality scores as currently reported are a lagging indicator. Several suppliers with acceptable quality scores on paper were associated with above-average production defect rates, suggesting that the quarterly manual scoring process misses problems that are already showing up on the factory floor. The pipeline flags suppliers with an average defect rate above five percent as high risk and suppliers with on-time delivery below ninety percent as below target, producing a tiered risk classification of LOW, MEDIUM, and HIGH for each supplier.

This scorecard gives the procurement team an objective, continuously updated view of supplier risk rather than a static quarterly snapshot. Suppliers classified as HIGH risk across both delivery and quality dimensions represent the clearest candidates for corrective action plans or replacement sourcing.

Inventory Risk and Reorder Point Optimization

The inventory analysis identified products that were at or below their reorder points at the most recent snapshot date. The pipeline calculated a stock gap metric for each SKU, which measures how far current inventory levels are from the reorder threshold, and flagged any product where stock had fallen to or below that point.

The results confirmed the CFO's concern about inventory imbalance. A significant share of SKUs were overstocked well beyond their reorder points, while a separate group had already crossed below their reorder thresholds and were at immediate risk of stockout. Each stockout event triggers between $15,000 and $50,000 in expedited freight costs, making this a quantifiable and avoidable expense.

The pipeline also calculated days-to-stockout estimates based on daily usage rates for at-risk products. This gives inventory planners a prioritized list of which products need immediate replenishment action and which have more time. Combined with supplier lead time data from the supplier performance dataset, these estimates allow planners to determine whether a standard purchase order will arrive in time or whether expedited sourcing is necessary.

Equipment Sensor Anomalies and Predictive Maintenance

The correlation analysis between sensor anomalies and production outcomes showed that shifts with abnormal temperature or vibration readings experienced both higher average downtime and higher defect rates compared to shifts with normal sensor readings. This confirms that machine health is directly linked to output quality, and that real-time sensor monitoring has genuine predictive value for both maintenance scheduling and quality assurance.

The real-time streaming pipeline operationalized this insight. Rather than discovering anomalies after the fact during a batch review, the system now detects them as they occur. The streaming job computes per-machine baselines from historical data, giving each of the eight hundred machines its own individualized alert thresholds based on its actual operating profile. This is a significant improvement over the fixed global thresholds used in earlier analysis.

During the live test run, the streaming pipeline detected multiple WARNING and CRITICAL alerts within the first two batches of data. MACH-0527 in FAC-02 triggered a temperature WARNING with a rolling average of 99.6 degrees Celsius against a machine-specific threshold of 96.91. MACH-0780, also in FAC-02, triggered a CRITICAL alert because both its temperature and vibration readings exceeded their respective thresholds simultaneously, indicating a high-risk condition.

These results demonstrate that the streaming pipeline can identify at-risk machines in near real-time, giving maintenance teams the window they need to intervene before a failure occurs rather than responding after an unplanned shutdown has already cost the company $50,000 per hour.

Scalability Discussion

Currently, our pipeline runs inside Docker containers on a single local machine. This works well for the current data volumes, but it would not hold up if data volume grew significantly. To understand where things would break, it helps to look at each part of the system one at a time.

If the data volume increased by ten times, the first thing that would struggle is HDFS. Our setup uses a single namenode which is responsible for keeping track of where all the files are stored. As more files get added, this one component has to handle more and more work until it eventually slows down or runs out of memory. The fix is to switch to a cloud storage service like AWS S3 or Google Cloud Storage. These services are built to store large amounts of data and scale automatically without any single component becoming a bottleneck.

Spark would also need to change at ten times the volume. Right now we run one Spark master and one worker on the same machine. At larger scales, Spark needs multiple machines working together. Moving to AWS EMR or Databricks would let Spark add or remove workers automatically depending on how much data needs to be processed, so the company only pays for what it actually uses.

Kafka would face similar issues. Our current setup uses a single broker, meaning all sensor data from machines flows through one point. If the number of machines grows, that single broker becomes a bottleneck and messages start to pile up. Adding more brokers and more topic partitions spreads the load across multiple machines. A managed service like AWS MSK or Confluent Cloud handles this automatically.

At one hundred times the current volume, bigger changes would be needed. Airflow is currently set up to run one task at a time on a single machine. At higher volumes, tasks would pile up and slow everything down. Switching to a setup that runs tasks across many machines at once, such as AWS MWAA, would fix this. Spark would also need some extra tuning to handle very large data joins more efficiently, using features like dynamic partition pruning and adaptive query execution.

The cloud version of our full pipeline would look like this: HDFS becomes AWS S3, Spark becomes AWS EMR or Databricks, Kafka becomes AWS MSK, and Airflow becomes AWS MWAA. AWS Step Functions could also be added to trigger pipeline runs automatically when new data arrives, rather than always relying on a fixed schedule.

The streaming part of our pipeline would benefit the most from moving to the cloud. AWS Kinesis can handle millions of sensor events per second, which is far more than our current single-broker Kafka setup can manage. Combined with AWS Athena for fast querying directly against S3, the system could handle real-time defect detection and equipment monitoring at any scale without needing a dedicated server running all the time.